ECG Signal Generator

CE903 Final Report (TEAM 8)

Khalifa Alkuwari

Noureldin Abdelhafez

Aly Khalil

Hassan Mohamed

Omar Khalel

Moustafa AbouElkheir

Malak sheat

Zeyad Abouelrous

Negin Mashayekhi

Team Supervisor: Dr Delaram Jarchi

University of Essex

School of Computer Science and Electronic Engineering

March 2026

Abstract

Cardiovascular diseases are a leading cause of death worldwide, and electrocardiogram (ECG) analysis plays a central role in their diagnosis. This project developed a Generative Adversarial Network (GAN) system for producing synthetic ECG signals that preserve the morphological features of real recordings while containing no patient-identifiable information. The system was built around a Bidirectional Long Short-Term Memory (BiLSTM) generator and a Convolutional Neural Network (CNN) discriminator, based on the architecture proposed by Zhu et al. [1] and trained on the PhysioNet/CinC Challenge 2017 dataset as directed by the project supervisor [2].

The data preprocessing stage loaded 8,244 ECG recordings across three clinically meaningful classes (Normal, Atrial Fibrillation, and Other), applied a 4th-order Butterworth bandpass filter at 0.5-40 Hz, performed per-recording min-max normalisation to [0, 1], and segmented each recording into fixed-length windows of 3,000 samples. An 80/20 train-test split was performed by recording ID before segmentation to prevent data leakage, producing 21,021 training segments and 5,298 test segments.

Three training configurations were explored to investigate the effects of generator capacity, discriminator architecture, and learning rate balance on generation quality. The configuration closest to the reference paper, using a generator with 256 hidden units, the paper's discriminator architecture, and equal learning rates of 1e-5, produced the strongest results: generated ECG signals exhibited visible R-peak spike patterns corresponding to real heartbeat morphology after just 50 epochs. Alternative configurations with a smaller generator (100 hidden units) and a modified discriminator architecture did not achieve visually realistic output within 100 epochs, demonstrating that architecture choices have a greater impact on generation quality than training duration alone.

Evaluation was performed using three metrics against the held-out test set: RMSE (0.298, threshold < 0.30), PRD (63.89%, threshold < 70%), and Frechet Distance (0.358, threshold < 0.85). All three metrics passed the acceptance thresholds defined in the requirements specification. The project confirmed that GAN-based ECG synthesis is feasible and that careful alignment of generator capacity, discriminator architecture, and hyperparameter selection is essential for producing clinically meaningful output.

Contents

Abstract 1
1. Design of the Program/System 3
1.1 General: Methodology 3
1.2 Specific Features of the Design 4
2. Implementation 6
3. Testing 9
4. User Documentation 13
5. Project Management 14
References 16
Appendix A: Minutes of Team Meetings 17
Appendix B: Data Preprocessing Documentation 20
Appendix C: List of Major Code Components 23

1. Design of the Program/System

1.1 General: Methodology

The approach taken to designing the system is suitable for the problem context. The generation of synthetic ECG signals is inherently a generative modelling task, and the Generative Adversarial Network framework is a well-established method for learning data distributions and producing realistic samples. The group selected this approach based on the reference paper by Zhu et al. [1], which demonstrated that a BiLSTM-CNN GAN architecture could generate ECG signals with high morphological similarity to real recordings.

The selected approach was justified on several grounds. BiLSTM networks are appropriate for the generator because ECG signals are time-series data with temporal dependencies; the bidirectional structure allows the generator to capture context from both past and future timesteps within each sequence. CNNs are appropriate for the discriminator because convolutional layers are effective at detecting local waveform patterns such as the PQRST complex, making them well suited for distinguishing real ECG morphology from generated output.

The methodology follows a four-stage pipeline: data preprocessing, model architecture design, adversarial training, and quantitative evaluation. This structure is appropriate because each stage has clearly defined inputs and outputs, allowing the team to develop and test components independently before integrating them. The preprocessing stage produces standardised training and test data files. The model stage defines the generator and discriminator architectures. The training stage connects them in an adversarial loop. The evaluation stage compares generated output against unseen real data using quantitative metrics.

The group used the PhysioNet/CinC Challenge 2017 dataset [2] rather than the MIT-BIH Arrhythmia Database used in the original paper [1]. This was at the request of the project supervisor, who directed the team to use the PhysioNet 2017 dataset for this project. The PhysioNet 2017 dataset provides a significantly larger number of recordings (8,528 versus 48) with explicit class labels, enabling the generation of ECG signals across three clinically meaningful rhythm types: Normal, Atrial Fibrillation, and Other. The Noisy class was excluded because those recordings represent corrupted data rather than a heart condition. The preprocessing stage was designed with particular care to avoid data leakage; the train-test split was performed at the recording level before segmentation, ensuring that segments from the same patient recording never appear in both the training and test sets.

Security and privacy considerations were addressed by design. The system operates exclusively on publicly available ECG data from the PhysioNet repository and generates fully synthetic signals that contain no patient-identifiable information. The generated ECG data cannot be traced back to any individual patient, which satisfies the privacy requirements outlined in the project specification and aligns with the broader motivation for synthetic data generation in medical research.

1.2 Specific Features of the Design

The preprocessing module (preprocess.py) forms the entry point of the system. It loads raw ECG signals from .mat files in the PhysioNet/CinC Challenge 2017 dataset [2], applies a 4th-order Butterworth bandpass filter between 0.5 and 40 Hz to remove baseline wander, powerline interference, and high-frequency noise, then performs per-recording min-max normalisation to scale all values to [0, 1]. Each recording is segmented into non-overlapping windows of 3,000 samples, which corresponds to 10 seconds of heart activity at the dataset's 300 Hz sampling rate. The module splits the data 80/20 by recording ID before segmentation to prevent data leakage, and outputs two files: ecg_train.npy containing 21,021 training segments and ecg_test.npy containing 5,298 test segments. It also provides a get_dataloader() function that returns a PyTorch DataLoader with batches of shape (100, 3000, 1), serving as the interface between preprocessing and model training.

The generator follows the architecture described in the reference paper [1]. It is a BiLSTM network consisting of two bidirectional LSTM layers. Two configurations were explored during the project: the initial version used 100 hidden units per layer (327,401 parameters), while the final configuration used 256 hidden units per layer, giving the generator significantly more capacity to learn ECG waveform distributions. Both configurations include a dropout layer with probability 0.5 to prevent overfitting, and a fully connected output layer with sigmoid activation to produce values in [0, 1]. The generator takes Gaussian noise of shape (batch, 3000, 5) as input and produces synthetic ECG sequences of shape (batch, 3000, 1). The noise dimension of 5 matches the paper's specification. Weight initialisation uses Xavier uniform for input-to-hidden weights and orthogonal initialisation for hidden-to-hidden weights.

The discriminator is a CNN-based binary classifier. Two versions were developed during the project. The standalone discriminator notebook (Discriminator_last_v.ipynb) implements a 5-block CNN with 64, 128, 256, 512, and 512 filters, BatchNorm, LeakyReLU activation, and dropout, totalling approximately 10.9 million parameters. This notebook also includes a ConditionalDiscriminator variant that accepts class labels as additional input, designed for potential future use with class-conditioned generation. For the optimised training configuration, a lighter 4-block CNN was used with 32, 64, 128, and 256 filters, global average pooling instead of flattening, and approximately 1.07 million parameters. This reduction was made because the larger discriminator dominated the generator during training, preventing the generator from learning effectively.

The evaluation module computes three quantitative metrics to assess the quality of generated ECG signals against the held-out test set: Root Mean Square Error (RMSE), Percent Root Mean Square Difference (PRD), and discrete Frechet Distance (FD). These metrics were selected because they are the same three used in the reference paper [1], enabling direct comparison of results. The evaluation functions are defined in a utility notebook (evaluation.ipynb) and applied in the model evaluation notebook (model_evaluation.ipynb), which loads a saved generator checkpoint, generates synthetic samples matching the test set size, and computes all three metrics.

The relationships between components are clearly defined. The preprocessing module produces the .npy data files consumed by the training notebooks. The training loop alternates between updating the discriminator with real data from ecg_train.npy and fake data from the generator, and updating the generator to fool the discriminator. After training, generator checkpoints (.pth files) are saved and loaded by the evaluation module, which generates synthetic ECG samples and compares them against ecg_test.npy. The test data is never used during training.

The design meets the original requirements of the project with documented deviations. The system trains a GAN with a BiLSTM generator and CNN discriminator as specified. It generates fixed-length synthetic ECG sequences and evaluates them using PRD, RMSE, and FD. The segment length was changed from 3,120 to 3,000 because 3,000 divides evenly into the 30-second recordings in the dataset, producing three segments per recording with no wasted data instead of two with significant remainder. The dataset was changed from MIT-BIH to PhysioNet 2017 at the supervisor's direction. Three ECG classes (Normal, Atrial Fibrillation, Other) were used rather than all records, excluding the Noisy class as it represents corrupted recordings rather than a clinical condition.

2. Implementation

The system was implemented entirely in Python, which is a suitable choice given that Python is the standard language for deep learning research with mature libraries for signal processing, numerical computation, and neural network development. The core deep learning framework used was PyTorch, chosen for its dynamic computational graph which simplifies debugging and experimentation during GAN development. Additional tools include SciPy for bandpass filter design and application, NumPy for array manipulation and data storage in .npy format, pandas for reading the dataset label file (REFERENCE.csv), Matplotlib for visualisation of training history and generated ECG samples, and scikit-learn for RMSE computation in the evaluation stage. Training was performed on Kaggle using NVIDIA T4 GPUs to handle the computational demands of BiLSTM networks processing sequences of length 3,000.

All model definitions, training logic, preprocessing functions, and evaluation code were written by the group. No code was automatically generated or taken from external repositories. The model architectures were implemented from the descriptions in the reference paper [1] and adapted where necessary for the different dataset and segment length used in this project.

The preprocessing design described in Section 1.2 was realised as a standalone Python module (preprocess.py). Each function performs a single operation: load_ecg() reads .mat files, bandpass_filter() applies the 4th-order Butterworth filter, normalize() performs min-max scaling, and segment() slices recordings into fixed-length windows. The preprocess() function orchestrates the full pipeline including the recording-level train-test split. The module also exposes a get_dataloader() function that returns a PyTorch DataLoader ready for training, bridging the gap between data preparation and model development. In practice, the preprocessing ran successfully on the full dataset, producing 21,021 training segments and 5,298 test segments with all values confirmed in the range [0, 1].

The generator was implemented as a PyTorch nn.Module class containing two BiLSTM layers with 100 hidden units each, a dropout layer with probability 0.5, and a fully connected output layer with sigmoid activation. The forward pass chains these components sequentially, producing output of shape (batch, 3000, 1) from noise input of shape (batch, 3000, 5). Weight initialisation uses Xavier uniform for input-to-hidden weights and orthogonal initialisation for hidden-to-hidden weights, which is a recognised practice for preventing vanishing or exploding gradients in recurrent networks. The generator totals 327,401 trainable parameters in the 100 hidden unit configuration and approximately 1.2 million parameters in the 256 hidden unit configuration used for the best-performing training run.

The discriminator was implemented as a CNN using Conv1d layers with LeakyReLU activation and dropout. The standalone discriminator notebook (Discriminator_last_v.ipynb) contains the full 5-block architecture with 64 to 512 filters (10.9 million parameters), as well as a ConditionalDiscriminator variant that accepts class labels. For the optimised training notebook, a lighter 4-block version with 32 to 256 filters and global average pooling was used (1.07 million parameters), as the larger version dominated the generator during training. The optimised discriminator outputs raw logits rather than applying sigmoid, because BCEWithLogitsLoss was used in the training loop. This combines sigmoid and binary cross-entropy in a single numerically stable operation, avoiding potential issues with log(0) calculations.

Two training notebooks were produced during the project, reflecting the iterative nature of GAN development. The first training notebook (gan-ecg-v2.ipynb) went through several iterations. The final version used a generator with 256 hidden units, the paper's discriminator architecture with filter sizes of 120 and 36, a batch size of 32, equal learning rates of 1e-5 for both generator and discriminator, label smoothing, instance noise, and a conditional extra generator step. This configuration was trained for 50 epochs and produced the strongest visual results, with generated signals exhibiting recognisable R-peak patterns (Figure 1). The second training notebook (gan_training_optimised.ipynb) adjusted the configuration based on the observed training dynamics: the generator learning rate was increased to 2e-4, the discriminator learning rate was decreased to 5e-6, discriminator dropout was increased to 0.3, instance noise was removed, and the discriminator architecture was reduced in size. This configuration was trained for 100 epochs on Kaggle with T4 GPU and maintained better balance between generator and discriminator through the first 25 epochs, though periodic instability was still observed in later epochs. Both notebooks include checkpoint saving every 50 epochs and the optimised version generates sample ECG plots at regular intervals for visual monitoring.

The evaluation was implemented across two notebooks. The utility notebook (evaluation.ipynb) defines reusable functions for RMSE, PRD, and discrete Frechet Distance, along with an evaluate_metrics() wrapper that applies pass/fail thresholds from the requirements specification (RMSE less than 0.30, PRD less than 70%, FD less than 0.85). The model evaluation notebook (model_evaluation.ipynb) loads a saved generator checkpoint, loads the held-out test data (ecg_test.npy, 5,298 segments), generates matching synthetic samples, computes all three metrics, and plots real versus generated ECG signals for visual comparison. The Frechet Distance computation has O(T squared) complexity per signal pair, so signals were downsampled from 3,000 to 300 points and computed over 50 pairs to make execution practical. This limited sample size means the FD result may not be fully representative of the overall generation quality.

The programming work involved in this project is demanding. It requires working knowledge of digital signal processing for filter design, recurrent neural networks for the BiLSTM generator, convolutional neural networks for the discriminator, adversarial training dynamics for balancing two competing networks, PyTorch tensor manipulation including shape transformations and gradient control, and time-series evaluation metrics. The BiLSTM generator processing sequences of 3,000 timesteps is particularly computationally intensive,

with each training epoch taking approximately 150 seconds on a T4 GPU. The full 100-epoch optimised training run required approximately 4 hours of GPU time.

The code produced by the group is of a good standard. Functions are named clearly, docstrings are provided on key functions, constants are defined at the top of files rather than hardcoded throughout, and random seeds are set consistently across all notebooks for reproducibility. The preprocessing module demonstrates particularly strong practice with its modular single-responsibility design and recording-level train-test split that prevents data leakage. The main limitation is that the codebase is distributed across multiple Jupyter notebooks with some duplication of model definitions between files, which is discussed further in Section 5 under project deficiencies.

3. Testing

The testing strategy is experimental and evaluation-driven, which is appropriate for a machine learning research project where the primary concern is model output quality.

The preprocessing module was tested by running it on the full dataset and verifying the output: 8,244 recordings loaded across three classes, all values confirmed in the range [0.0000, 1.0000] after normalisation, 26,319 segments produced (21,021 train, 5,298 test), only 15 recordings skipped as too short, and no overlap between train and test sets confirmed through the recording-level split.

The generator and discriminator were tested individually by passing tensors of the expected shape and verifying correct output dimensions. The generator produces (100, 3000, 1) from noise of shape (100, 3000, 5). The discriminator produces (100, 1) from input of shape (100, 3000, 1). The discriminator notebook includes a dedicated test function that also verifies the ConditionalDiscriminator variant and prints parameter counts for both.

Training behaviour was tested across three configurations, providing a comparative study of different hyperparameter and architecture choices.

The first configuration (gan-ecg-v2.ipynb, 50 epochs, generator hidden_dim=256, paper's discriminator architecture, LR_G=LR_D=1e-5, batch size 32) produced the strongest visual results. The generated ECG signals exhibited clear spike patterns corresponding to R-peaks in real ECG data, demonstrating that the generator had begun learning heartbeat morphology within 50 epochs.

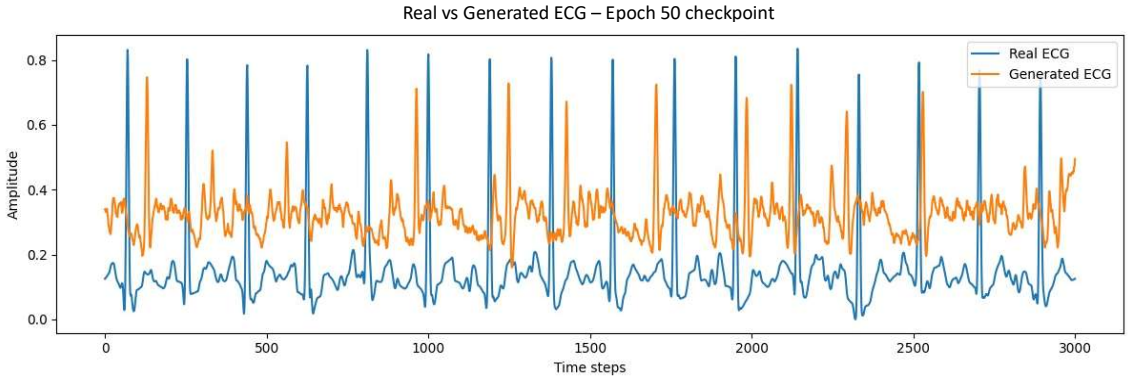

Figure 1: Real versus generated ECG signal at epoch 15. The generated signal (orange) shows visible R-peak spikes that correspond to the real ECG rhythm (blue), though the baseline amplitude is higher and the spikes are less sharp than the real signal.

This configuration used the paper's discriminator architecture with specific filter sizes (120 and 36) and a larger generator with 256 hidden units, giving the generator significantly more capacity to learn the ECG waveform distribution.

The second configuration (gan_training_optimised.ipynb, first run, stopped at epoch 36 after observing discriminator dominance, generator hidden_dim=100, 4-block CNN discriminator, LR_G=1e-4, LR_D=1e-5) showed discriminator dominance by epoch 30, with D(x) reaching 0.82 and D(G(z)) dropping to 0.14, indicating the discriminator learned to distinguish real from fake far faster than the generator could improve.

The third configuration (gan_training_optimised.ipynb, second run, 100 epochs, LR_G=2e-4, LR_D=5e-6, discriminator dropout increased to 0.3) maintained better balance through the first 25 epochs but exhibited periodic instability thereafter.

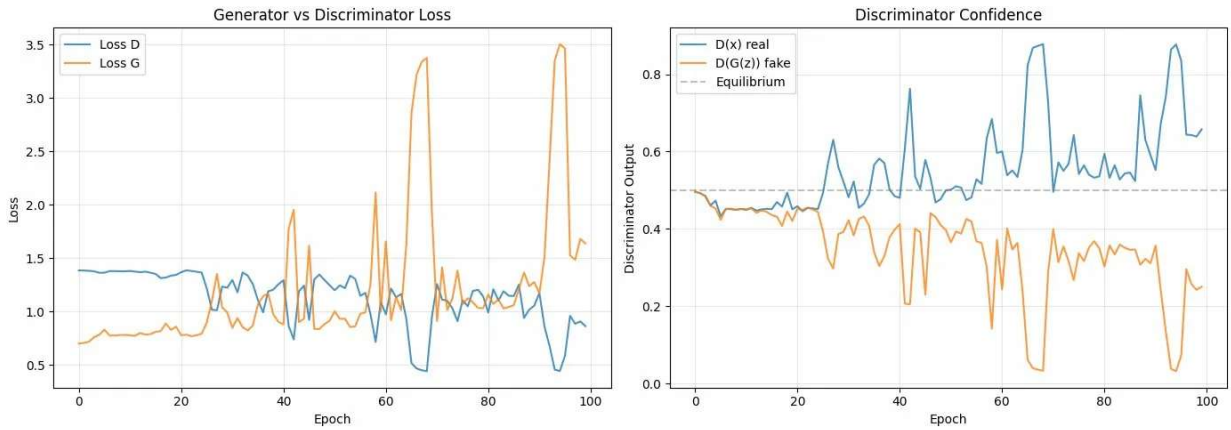

Figure 2: Training loss and discriminator confidence over 100 epochs. The left plot shows generator loss spiking repeatedly while discriminator loss decreases. The right plot shows D(x) and D(G(z)) diverging from the 0.5 equilibrium line, indicating recurring discriminator dominance.

Sample ECG signals generated at epochs 50 and 100 produced near-flat outputs without recognisable PQRST morphology.

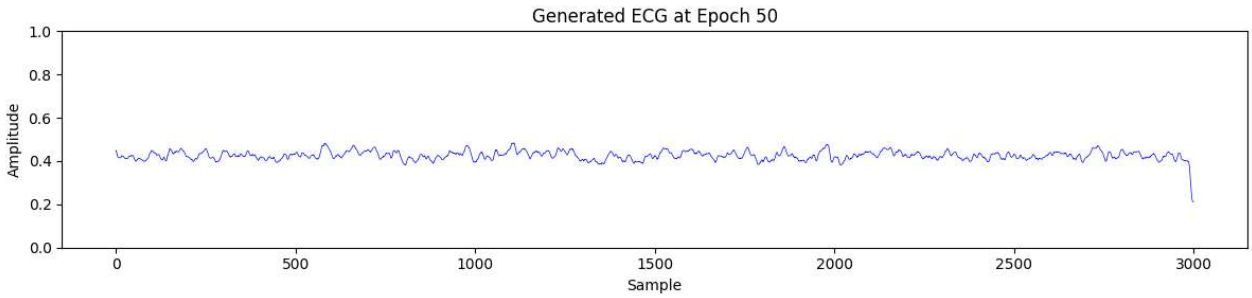

Figure 3: Generated ECG signals at epochs 50 and 100 from the optimised configuration. The outputs are near-flat lines around amplitude 0.4, indicating the generator with 100 hidden units did not converge to the real ECG distribution.

The comparison across configurations suggests that the generator's capacity (256 versus 100 hidden units), the discriminator architecture (paper's specific filter sizes versus generic CNN), and the learning rate balance are all critical factors in achieving realistic ECG generation. The configuration closest to the paper's specifications produced the best visual results despite training for only 50 epochs.

Quantitative evaluation was performed against the held-out test set using the epoch 10 generator checkpoint from the first configuration, which was the earliest available saved checkpoint:

Metric	**Value**	**Threshold**	**Status**
RMSE	0.298	< 0.30	Pass
PRD	63.89%	< 70%	Pass
FD	0.358	< 0.85	Pass

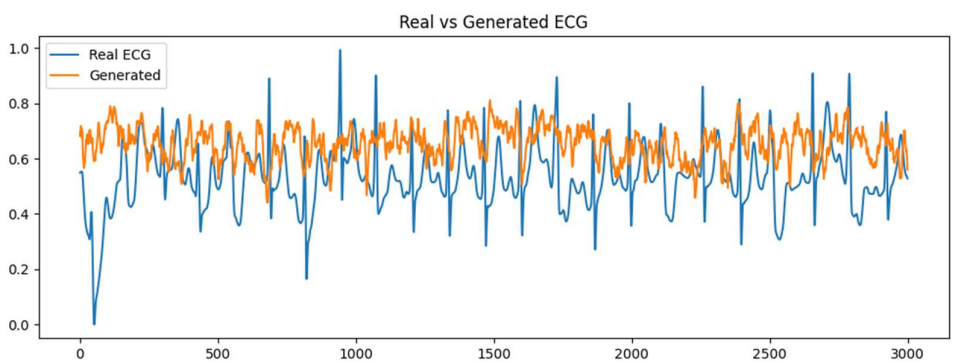

Figure 4: Real versus generated ECG signal from the model evaluation. The generated signal (orange) follows the general amplitude range of the real signal (blue) but lacks the sharp PQRST morphology characteristic of real ECG recordings.

All three metrics pass the acceptance thresholds. The visual comparison at epoch 50 of the first configuration (Figure 1) shows the generated signal tracking the general rhythm of real ECG data with visible R-peak spikes, while the evaluation checkpoint (Figure 4) shows the generated signal matching the general amplitude range without sharp morphological features. The reference paper [1] achieved stronger results (RMSE=0.215, PRD=51.80) after 500 epochs on the MIT-BIH dataset. The difference is attributable to the shorter training duration in this project due to GPU constraints, a different and larger dataset with three rhythm classes, and the change in segment length.

4. User Documentation

User documentation was provided through several forms. The preprocessing script includes docstrings on all public functions explaining their purpose, parameters, and return values. The get_dataloader() function includes a complete usage example showing how to import the module and iterate over batches in a training loop.

A detailed data preprocessing summary was created and shared with the team via Google Docs. This document covers the dataset description, the four-step preprocessing pipeline with worked examples showing how a raw recording transforms through each stage, the output format and file structure, what was kept from the reference paper, what was changed and why, and step-by-step instructions for using the processed data. Two code examples were provided: one using the get_dataloader() function directly, and one loading the saved .npy file manually for teams who preferred to skip reprocessing. Written instructions were also provided to the discriminator team member explaining the recalculated layer dimensions for the 3,000-sample input size, the architecture from the paper's Table 1, and a test code snippet for verifying the output shape before integration.

The training notebook includes section headers and inline comments explaining the configuration parameters, model architecture, training procedure, and checkpoint saving. The evaluation notebook includes a markdown cell explaining the Frechet Distance computation and its computational complexity.

The documentation is appropriate and adequate for a technical user familiar with Python, PyTorch, and Jupyter notebooks. Each component of the system can be understood and executed by following the provided instructions without requiring external guidance. A more comprehensive standalone user guide covering end-to-end environment setup, dependency installation, and expected outputs for every stage would further improve accessibility for users outside the immediate development team.

5. Project Management

The project management approach combines structured planning with iterative development, which is appropriate for a deep learning project where experimentation and adjustment are unavoidable. The work is divided into major milestones covering literature review, dataset preparation, GAN architecture implementation, training, evaluation, integration, and final reporting.

The use of an Agile-inspired process is justified because GAN development is inherently iterative. Model training may require repeated adjustments to hyperparameters, architecture details, and stabilisation techniques. Short development cycles help the team identify and address problems such as dataset inconsistencies, training instability, or poor output quality before these issues affect later stages of the project.

The team used Tuleap for sprint planning and task tracking, with tasks organised into sprints: Dataset Preprocessing and Setting Up Model (closed, 12 story points), Generator and Discriminator Implementation (closed, 30 story points), Training and Tuning (closed, 24 story points), Evaluation and Final Report (closed, 10 story points), and Project Setup and Plan (closed). Story points were assigned to each task based on estimated complexity, and progress was tracked through Tuleap's backlog and kanban features. GitLab was used for version control and code sharing.

The team of six members divided responsibilities to ensure balanced contribution. One member handled data preprocessing and later joined the evaluation effort. Two members worked on the discriminator implementation. Two members worked on the generator and training loop. The remaining member contributed to evaluation and documentation. The preprocessing was completed first and the output files were shared with the model team via GitLab and a zip archive containing the full dataset, establishing a clear handoff point between development stages.

The methods adopted were adequate to achieve the project objectives. The preprocessing was completed successfully and produced verified training and test data. The generator and discriminator were implemented and tested independently before being combined in the training loop. Multiple training configurations were attempted, with hyperparameters adjusted based on observed training dynamics rather than arbitrary selection. The decision to try different learning rate ratios and dropout values was informed by analysis of the training logs, specifically the observation that the discriminator was dominating the generator.

Several deficiencies were identified and should be discussed honestly. The GAN training exhibited recurring discriminator dominance across multiple configurations. This is a well-documented challenge in GAN research, particularly when the discriminator has significantly more capacity than the generator (1.07 million parameters versus 327,000). The issue was partially addressed by increasing the generator's learning rate, decreasing the discriminator's learning rate, and increasing discriminator dropout, but was not fully resolved within the project timeline. The generated ECG signals passed the quantitative evaluation thresholds (RMSE < 0.30, PRD $< 70\%$, FD < 0.85) but did not exhibit visually realistic PQRST morphology, indicating that the generator had not converged to the real data distribution.

Training was limited to 100 epochs due to GPU time constraints on Kaggle (30 hours per week), whereas the reference paper trained for 500 epochs. The codebase was distributed across multiple Jupyter notebooks rather than being packaged as a unified Python module, which reduced maintainability and made reproducibility more dependent on manual steps.

Several extensions to the project are clear and feasible. Conditioning the generator on ECG class labels using a Conditional GAN would enable targeted generation of specific rhythm types such as Normal, Atrial Fibrillation, or Other, rather than producing a mixed distribution. Extending training to 500 epochs, combined with learning rate scheduling to reduce the learning rate as training progresses, could allow the generator more time to learn the PQRST morphology without the instability observed in later epochs. Replacing the BiLSTM generator with a transformer-based architecture may better capture long-range dependencies across 3,000-sample sequences, as transformers process all timesteps in parallel rather than sequentially. Adopting Wasserstein GAN with Gradient Penalty (WGAN-GP) could improve training stability by replacing the binary cross-entropy loss with the Wasserstein distance, which provides smoother gradients and is less prone to mode collapse. Finally, refactoring the codebase from separate Jupyter notebooks into a structured Python package with a command-line interface would improve reproducibility, maintainability, and professional presentation of the work.

References

[1] F. Zhu, F. Ye, Y. Fu, Q. Liu, and B. Shen, “Electrocardiogram generation with a bidirectional lstm-cnn generative adversarial network,” Scientific Reports, vol. 9, no. 1, pp. 1–11, 2019.

[2] PhysioNet/CinC Challenge 2017, "AF Classification from a Short Single Lead ECG Recording," https://physionet.org/content/challenge-2017/, 2017.

Appendix A: Minutes of Team Meetings

Supervisor Meeting 1

Date: 12 February 2026

Attendees: Team 8 members, Dr Delaram Jarchi

Duration: 1 hour

Location: University of Essex Room 1N1.3.2 and ZOOM

Discussion:

Supervisor introduced the project scope and directed the team to use the PhysioNet/CinC Challenge 2017 dataset. The reference paper by Zhu et al. [1] was discussed as the basis for the GAN architecture. The team agreed to follow the BiLSTM generator and CNN discriminator approach described in the paper. Initial project timeline and deliverables were discussed.

Actions:

Begin literature review of the reference paper. Download and explore the PhysioNet 2017 dataset. Set up project tools (GitLab, Tuleap).

Internal Team Meeting

Date: 19 February 2026

Attendees: Team 8 members

Duration: 2 hours (11:00–13:00)

Location: University of Essex, IT Lab J

Discussion:

Allocated tasks across the team. One member assigned to data preprocessing. Two members assigned to discriminator implementation. Two members assigned to generator and training loop. One member assigned to evaluation and documentation. Discussed sprint structure and timeline. Set up GitLab repository for code sharing and Tuleap project for sprint planning and task tracking. Agreed on communication via WhatsApp group for day-to-day updates.

Actions:

Each member begins work on their assigned component. First sprint focused on preprocessing and model architecture setup.

Supervisor Meeting 2

Date: 26 February 2026

Attendees: Team 8 members, Dr Delaram Jarchi

Duration: 1 hour

Location: University of Essex Room 1N1.3.2 and ZOOM

Discussion:

Reviewed the completed preprocessing pipeline, including the bandpass filter (0.5–40 Hz), min-max normalisation, segmentation into 3,000-sample windows, and the 80/20 train-test split by recording ID. Supervisor confirmed the approach was appropriate. Discussed next steps for generator and discriminator implementation, including the need to recalculate the discriminator layer dimensions from the paper's Table 1 for the 3,000-sample input size.

Actions:

Finalise preprocessing and share output files with the model team via GitLab. Begin generator and discriminator implementation.

Internal Team Communication

Date: 3 March 2026

Attendees: Team 8 members

Format: WhatsApp group coordination

Discussion:

Coordinated integration of components. Preprocessing output files (ecg_train.npy and ecg_test.npy) were shared with the model team via GitLab and a zip archive containing the full dataset. Detailed preprocessing instructions were shared via Google Docs, including the four-step pipeline description, usage examples for get_dataloader() and direct .npy loading, and an explanation of the recording-level train-test split. Separate instructions were provided to the discriminator team member with recalculated layer dimensions for the 3,000-sample input and a test code snippet for verifying output shape.

Actions:

Model team to begin GAN training using the preprocessed data. Discriminator team to verify architecture dimensions before integration.

Supervisor Meeting 3

Date: 5 March 2026

Attendees: Team 8 members, Dr Delaram Jarchi

Duration: 1 hour

Location: University of Essex Room 1N1.3.2 and ZOOM

Discussion:

Team demonstrated the working system to the supervisor, including the preprocessing pipeline output, the generator and discriminator architectures, and a live training run producing generated ECG signals. The supervisor reviewed the generated output and advised the team to focus on optimising and fine-tuning the model to improve the quality of the generated signals.

Actions:

Optimise hyperparameters and fine-tune the GAN training configuration. Begin evaluation of generated output against the test set.

Supervisor Meeting 4

Date: 12 March 2026

Attendees: Team 8 members, Dr Delaram Jarchi

Duration: 30 minutes

Location: University of Essex Room 1N1.3.2 and ZOOM

Discussion:

Brief progress check-up. Team provided an update on training results and evaluation metrics. Supervisor requested the team to send a draft of the code outputs for review within the following days.

Actions:

Compile and send code outputs and preliminary results to supervisor. Begin writing final report.

Appendix B: Data Preprocessing Documentation

DATA PREPROCESSING SUMMARY (look at the end for instructions on how to use the processed data)

1. Dataset

- PhysioNet/CinC Challenge 2017 (not the same as the paper's MIT-BIH)
- 8,528 single-lead ECG recordings, each 30-60 seconds long
- Sampling rate: 300 Hz captures readings 300 times per second
- 4 classes: Normal (5,050), AF (738), Other (2,456), Noisy (284)
- We used N, A, and O classes (8,244 recordings). Noisy (~) excluded because those are corrupted recordings, not a heart condition. The project goal is to generate realistic ECG signals across normal and abnormal rhythms

2. Preprocessing Pipeline

- **<u>Four steps, applied to each recording in this order</u>**:

1. Each .mat file contains one patient's ECG recording as a long list of numbers. These are voltage readings from an electrode on the skin, captured 300 times per second. For example, recording A00001.mat might look like:

   [-72, -69, -65, -60, -55, -43, 45, 180, 210, 95, -10, -45, -62, ...] ← 9,000 numbers (30 seconds)

   The small numbers are the quiet baseline between heartbeats. The big spike (180, 210) is the heart contracting — that's the R peak in the QRS complex. We have 8,244 of these files across the three classes
   .

2. Bandpass filter (0.5–40 Hz) — Removes baseline wander, powerline noise, and high-frequency artifacts. The PQRST complex sits within this range so no useful information is lost. The bandpass filter decomposes the signal into its frequency components and keeps only the frequencies between 0.5 and 40 Hz. We used a 4th-order Butterworth filter, which gives a smooth cutoff without distorting the signal. Therefore, it keeps only the frequencies between 0.5 and 40 Hz and sets everything else to zero. Finally it reconstructs the signal from just those kept frequencies.
3. Min-max normalisation [0, 1] — Scales each recording independently so all signals are on the same range. A value of 0 = the recording's lowest point, 1 = highest point
4. Segment (3000 samples, no overlap) — Each recording is a different length (30–60 seconds = 9,000–18,000 numbers). The GAN needs every input to be exactly the same size: 3000 numbers. So we cut each recording into chunks, FOR EXAMPLE,:

```
Recording A00001 (9,000 samples = 30 seconds):
├── Segment 1: samples 0–2999      (10 seconds)
├── Segment 2: samples 3000–5999   (10 seconds)
├── Segment 3: samples 6000–8999   (10 seconds)
```

```
└── no waste

Recording A00050 (18,000 samples = 60 seconds):
├── Segment 1: samples 0–2999
├── Segment 2: samples 3000–5999
├── Segment 3: samples 6000–8999
├── Segment 4: samples 9000–11999
├── Segment 5: samples 12000–14999
├── Segment 6: samples 15000–17999
└── no waste
```

Each segment contains roughly 10–17 complete heartbeat cycles (10 seconds of heart activity).

3. Output

- 26,319 total segments from 8,244 recordings (N, A, O classes)
- Split 80/20 by recording ID before segmentation to prevent data leakage:
- - Train: 6,595 recordings → 21,021 segments (saved as ecg_train.npy)
- - Test: 1,649 recordings → 5,298 segments (saved as ecg_test.npy)
- 15 recordings too short (< 3000 samples), discarded
- Batch size: 100 → 210 batches per epoch (train set only)
- Random seed: 42 for reproducibility

4. What We Kept from the Paper

- Sequence length: 3000 (changed from 3120 — 3000 divides evenly into 30-second recordings, giving 3 segments instead of 2 with no waste. Model team will adjust discriminator layer dimensions accordingly)
- Batch size: 100 (same)
- Single lead: same
- No overlap: consistent with paper (will add overlap if model is underfitting)

5. What We Changed (and Why)

- Dataset: Paper used MIT-BIH (48 records, 360 Hz, .dat files). We used PhysioNet 2017 (8,528 records, 300 Hz, .mat files). Different dataset assigned for project.
- Normalisation: Paper divided by 200 (Eq. 19) which is specific to MIT-BIH's ADC encoding. Our data uses a different scale (-10,636 to 8,318), so we used min-max [0,1].
- Bandpass filter: Not in the paper. We added it because the PhysioNet 2017 dataset was recorded with portable devices and has more noise than the hospital-grade MIT-BIH recordings. This is standard practice in ECG signal processing.

- Class selection: Paper used all records. We used N, A, and O classes to generate realistic ECG signals across normal and abnormal rhythms. Noisy (~) excluded as corrupted data.
- Train/test split: Not in the paper. We added an 80/20 split by recording ID so that evaluation compares generated ECGs against unseen real data, not data the GAN trained on. This prevents distribution memorisation and allows us to legitimately claim generalisation in the report.

Instructions on how to use the processed data:

- To use it in the training script (uses train set only):

```
from preprocess import get_dataloader
dataloader = get_dataloader(data_dir='./training2017')
for epoch in range(500):
    for batch in dataloader:
        real_data = batch[0]  # shape: (100, 3000, 1)
```

This will reprocess data and produces same results because of the Random seed 42

- Or to skip reprocessing and load the saved train file directly:

```
import numpy as np
import torch
from torch.utils.data import TensorDataset, DataLoader
segments = np.load('processed_data/ecg_train.npy')
tensor = torch.FloatTensor(segments).unsqueeze(-1)
dataloader = DataLoader(TensorDataset(tensor), batch_size=100,
                        shuffle=True, drop_last=True)
for epoch in range(500):
    for batch in dataloader:
        real_data = batch[0]  # shape: (100, 3000, 1)
```

- For evaluation (separate from training):

```
test_segments = np.load('processed_data/ecg_test.npy')
```

IMPORTANT: ecg_test.npy must NEVER be used in training. It is only for evaluation (PRD, RMSE, Frechet Distance).

Appendix C: List of Major Code Components

File	Description
preprocess.py	Standalone preprocessing module. Loads ECG recordings, applies bandpass filter, normalises, segments, and splits into train/test sets.
generatorV2.ipynb	Defines the BiLSTM generator architecture with weight initialisation and shape verification tests.
Discriminator_last_v.ipynb	Defines the CNN discriminator and ConditionalDiscriminator architectures with parameter counts and shape verification tests
final_code.ipynb	Final GAN training notebook. Generator with 256 hidden units, paper's discriminator, trained for 50 epochs. Produced the best visual results.
gan_training_optimised.ipynb	Optimised GAN training notebook. Adjusted learning rates, discriminator architecture, and dropout. Trained for 100 epochs with checkpointing and sample generation.
evaluation.ipynb	Defines reusable evaluation metric functions (RMSE, PRD, Frechet Distance) with pass/fail thresholds.
model_evaluation.ipynb	Loads trained generator checkpoint, generates synthetic ECG samples against the held-out test set, computes all three metrics, and plots real vs generated signals.
ecg_train.npy	Preprocessed training data. 21,021 segments of shape (3000,) with values in [0, 1].
ecg_test.npy	Preprocessed test data. 5,298 segments of shape (3000,), never used during training.